

Alumno: Marco Antonio González Gutiérrez

Materia: introducción a las ciencias de datos

Profesor: Romero Sierra Jaime Alejandro

Fecha de entrega: 28/11/2025

GitHub:

<https://github.com/gonzalezgutierrez429star/>

Limpieza_De_Base_de_datos

1 Descripción inicial de la base de datos Fuente y contexto de la base de datos

La base de datos seleccionada contiene información sobre reclamaciones de seguros de automóviles. Este dataset incluye datos demográficos, información del vehículo, detalles del accidente y características de la póliza.

Descripción general del contenido El

dataset original cuenta con:

- **Registros:** 19,018 registros
- **Columnas:** 33 columnas
- **Período:** Datos de los años 1994-1996

Dataset después de la limpieza

- **Registros:** 4783 registros
- **Columnas:** 33 columnas

Make	MarcaVehiculo	Marca del Vehiculo
AccidentArea	AreaAccidente	Area donde ocurrió el accidente
DayOfWeekClaimed	DiaReclamo	Dia de la semana del reclamo
MonthClaimed	MesReclamo	Mes del reclamo
WeekOfMounthClaimed	SemanaReclamo	Semana del reclamo
Sex	Genero	Genero del asegurado
MarialStatus	EstadoCivil	Estado civil del asegurado
Age	Edad	Edad del conductor
Fault	Culpa	Parte culpable del accidente
PolicyType	TipoPoliza	Tipo de poliza

VehicleCategory	CategoriaVehiculo	Categoria del vehiculo
VehiclePrice	PrecioVehiculo	Precio del vehículo
FraudFound_P	FraudeDetectado	Indicador del fraude detectado
PolicyNumber	NumeroPoliza	Numero de póliza
RepNumber	NumeroRepresentante	Numero de representante
Deductible	Deductible	Monto deducible
DriverRating	CalificacionConductor	Calificacion del conductor
Days_Policy_Accident	DiasPolizaAccidente	Dias desde la póliza del accidente
Days_Policy_Claim	DiasPolizaReclamo	Dias desde la póliza de reclamo
PastNumberOfClaims	NumeroReclamosAnteriores	Numero de reclamos importantes
AgeOfVehicle	EdadVehiculo	Edad del vehículo
AgeOfPolicyHolder	EdadAsegurado	Edad del asegurado (en rangos)
PoliceReportFiled	ReportePolicial	Si se archivó el reporte policial
WitnessPresent	TestigoPresente	Si había testigos presentes
AgentType	TipoAgente	Tipo de agente
NumbersOfSuppliments	NumeroSuplementos	Numero de suplementos
AddressChange_Claim	CambioDireccionReclamo	Cambio de dirección antes del reclamo
NumbersOfCars	NumeroVehiculos	Número de vehículos
Year	Año	Año del registro
BasePolicy	PolizaBase	Poliza base

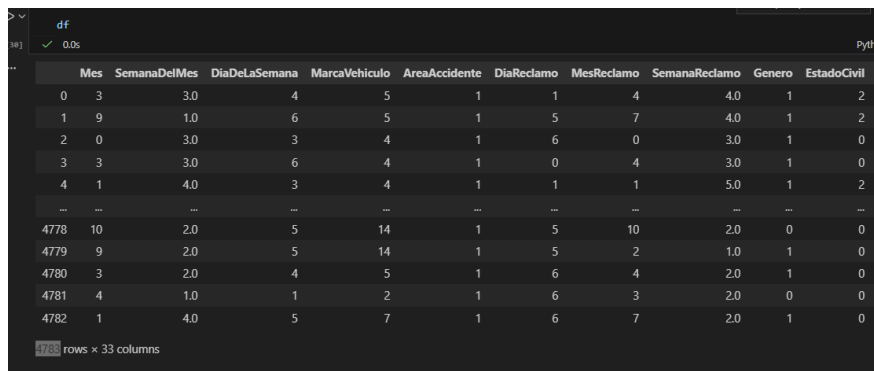
```
import pandas as pd
import numpy as np

df = pd.read_csv("C:/Users/anton/OneDrive/Documentos/limpieza/fraud_oracle.csv")
df
```

[36] ✓ 0.0s Python

	Month	WeekOfMonth	DayOfWeek	Make	AccidentArea	DayOfWeekClaimed	MonthClaimed	WeekOfMonthClaimed	Sex	MaritalStatus	...	AgeOfVehicle	AgeOfPolicyHolder	PoliceRe
0	Dec	5	Wednesday	Honda	Urban	Tuesday	Jan		1	Female	Single	...	3 years	26 to 30
1	Jan	3	Wednesday	Honda	Urban	Monday	Jan		4	Male	Single	...	6 years	31 to 35
2	Oct	5	Friday	Honda	Urban	Thursday	Nov		2	Male	Married	...	7 years	41 to 50
3	Jun	2	Saturday	Toyota	Rural	Friday	Jul		1	Male	Married	...	more than 7	51 to 65
4	Jan	5	Monday	Honda	Urban	Tuesday	Feb		2	Female	Single	...	5 years	31 to 35
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
15415	Nov	4	Friday	Toyota	Urban	Tuesday	Nov		5	Male	Married	...	6 years	31 to 35
15416	Nov	5	Thursday	Pontiac	Urban	Friday	Dec		1	Male	Married	...	6 years	31 to 35
15417	Nov	5	Thursday	Toyota	Rural	Friday	Dec		1	Male	Single	...	5 years	26 to 30
15418	Dec	1	Monday	Toyota	Urban	Thursday	Dec		2	Female	Married	...	2 years	31 to 35
15419	Dec	2	Wednesday	Toyota	Urban	Thursday	Dec		3	Male	Single	...	5 years	26 to 30

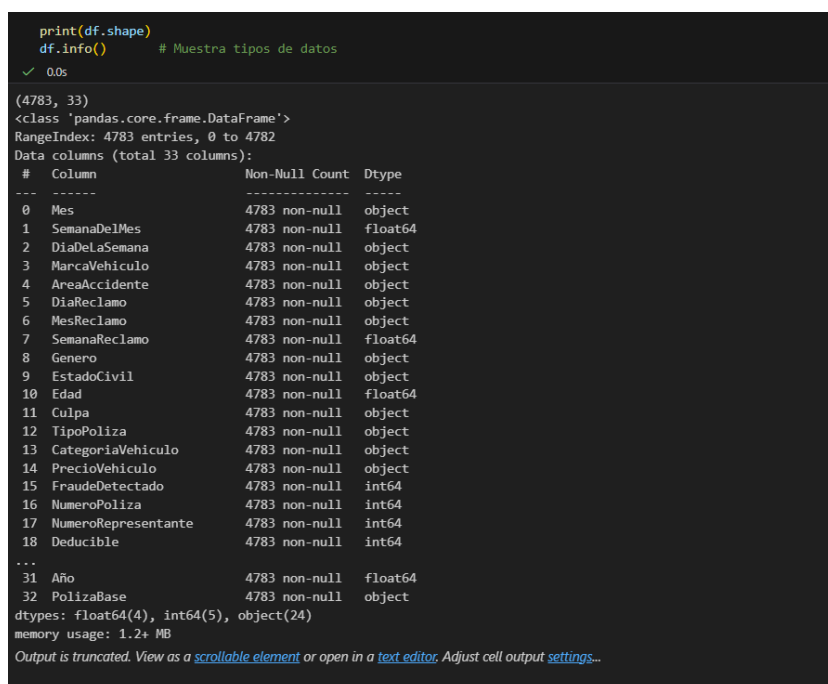
DESPUES DE LA LIMPIEZA



The screenshot shows a Jupyter Notebook interface with a cell containing the variable 'df'. The output is a preview of a DataFrame with 4783 rows and 33 columns. The first few rows are displayed, showing columns: Mes, SemanaDelMes, DiaDeLaSemana, MarcaVehiculo, AreaAccidente, DiaReclamo, MesReclamo, SemanaReclamo, Genero, and EstadoCivil. The data appears to be related to insurance claims.

	Mes	SemanaDelMes	DiaDeLaSemana	MarcaVehiculo	AreaAccidente	DiaReclamo	MesReclamo	SemanaReclamo	Genero	EstadoCivil
0	3	3.0	4	5	1	1	4	4.0	1	2
1	9	1.0	6	5	1	5	7	4.0	1	2
2	0	3.0	3	4	1	6	0	3.0	1	0
3	3	3.0	6	4	1	0	4	3.0	1	0
4	1	4.0	3	4	1	1	1	5.0	1	2
...	...	...	...	...	...	...	...	...	...	...
4778	10	2.0	5	14	1	5	10	2.0	0	0
4779	9	2.0	5	14	1	5	2	1.0	1	0
4780	3	2.0	4	5	1	6	4	2.0	1	0
4781	4	1.0	1	2	1	6	3	2.0	0	0
4782	1	4.0	5	7	1	6	7	2.0	1	0

Para este trabajo se utilizó un modelo de machine learning basado en la regresión lineal. En la primera celda del notebook se importan las bibliotecas principales que se utilizarán en el análisis, incluyendo pandas, numpy, matplotlib y seaborn, que permiten la manipulación de datos y la creación de visualizaciones. Después de cargar el archivo correspondiente al dataset procesado, se muestra una vista preliminar de las primeras filas con el propósito de verificar que la información fue cargada correctamente y observar su estructura general.



The screenshot shows a Jupyter Notebook cell with the following code and output:

```
print(df.shape)
df.info() # Muestra tipos de datos
```

Output:

```
(4783, 33)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4783 entries, 0 to 4782
Data columns (total 33 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Mes                    4783 non-null   object
1   SemanaDelMes           4783 non-null   float64
2   DiaDeLaSemana          4783 non-null   object
3   MarcaVehiculo          4783 non-null   object
4   AreaAccidente          4783 non-null   object
5   DiaReclamo             4783 non-null   object
6   MesReclamo             4783 non-null   object
7   SemanaReclamo          4783 non-null   float64
8   Genero                 4783 non-null   object
9   EstadoCivil           4783 non-null   object
10  Edad                  4783 non-null   float64
11  Culpa                 4783 non-null   object
12  TipoPoliza            4783 non-null   object
13  CategoriaVehiculo     4783 non-null   object
14  PrecioVehiculo        4783 non-null   object
15  FraudeDetectado       4783 non-null   int64
16  NumeroPoliza          4783 non-null   int64
17  NumeroRepresentante    4783 non-null   int64
18  Deducible             4783 non-null   int64
...
31  Año                   4783 non-null   float64
32  PolizaBase            4783 non-null   object
dtypes: float64(4), int64(5), object(24)
memory usage: 1.2+ MB
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

En la segunda celda se obtiene una descripción básica del dataset utilizando `df.shape` para conocer la cantidad total de filas y columnas, junto con `df.info()` que permite identificar los tipos de datos presentes y detectar posibles valores faltantes. Esta celda funciona como un primer diagnóstico para comprender la estructura y calidad de la información antes de cualquier transformación adicional.

```
print("Análisis de Calidad de Datos")

duplicados = df.duplicated().sum()
print(f"Cantidad de filas duplicadas encontradas: {duplicados}")
if duplicados > 0:
    df = df.drop_duplicates()
    print("Duplicados eliminados.")
else:
    print("No se encontraron duplicados.")

nulos = df.isnull().sum()
print("\nValores nulos por columna:")
print(nulos[nulos > 0]) # Solo muestra las que tienen nulos.
```

✓ 0.0s

```
Análisis de Calidad de Datos
Cantidad de filas duplicadas encontradas: 0
No se encontraron duplicados.

Valores nulos por columna:
Series([], dtype: int64)
```

La tercera celda se enfoca en evaluar la calidad de los datos mediante la detección de duplicados y valores nulos. Aquí se calcula cuántos registros están repetidos y se revisa cada columna para identificar la presencia de valores faltantes. El propósito de esta parte del proceso es determinar si el dataset requiere limpieza adicional debido a problemas comunes como duplicados o datos incompletos.

```

print(df.describe()) # Para numéricas

print(df['FraudeDetectado'].value_counts())
print(df['MarcaVehiculo'].value_counts())

```

	SemanaDelMes	SemanaReclamo	Edad	FraudeDetectado \
count	4783.000000	4783.000000	4783.000000	4783.000000
mean	2.781936	2.708133	39.844031	0.056659
std	1.292901	1.258892	13.494826	0.231214
min	1.000000	1.000000	0.000000	0.000000
25%	2.000000	2.000000	30.000000	0.000000
50%	3.000000	3.000000	38.000000	0.000000
75%	4.000000	4.000000	48.000000	0.000000
max	5.000000	5.000000	80.000000	1.000000

	NumeroPoliza	NumeroRepresentante	Deducible	CalificacionConductor \
count	4783.000000	4783.000000	4783.000000	4783.000000
mean	7795.45369	8.473134	407.024880	2.507004
std	4450.29266	4.553513	42.417445	1.116913
min	2.000000	1.000000	300.000000	1.000000
25%	3928.50000	5.000000	400.000000	1.500000
50%	7881.00000	8.000000	400.000000	3.000000
75%	11654.50000	12.000000	400.000000	3.000000
max	15419.00000	16.000000	700.000000	4.000000

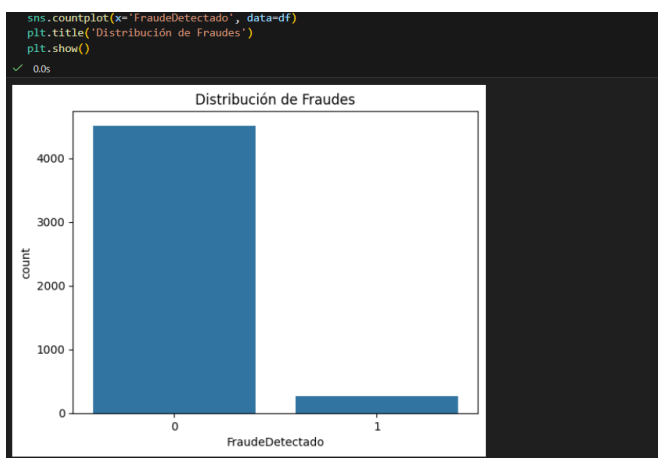

```

Año
count 4783.000000
mean 1994.882710
std 0.805564
min 1994.000000
...
BMW 6
Jaguar 3
Porsche 1
Name: count, dtype: int64

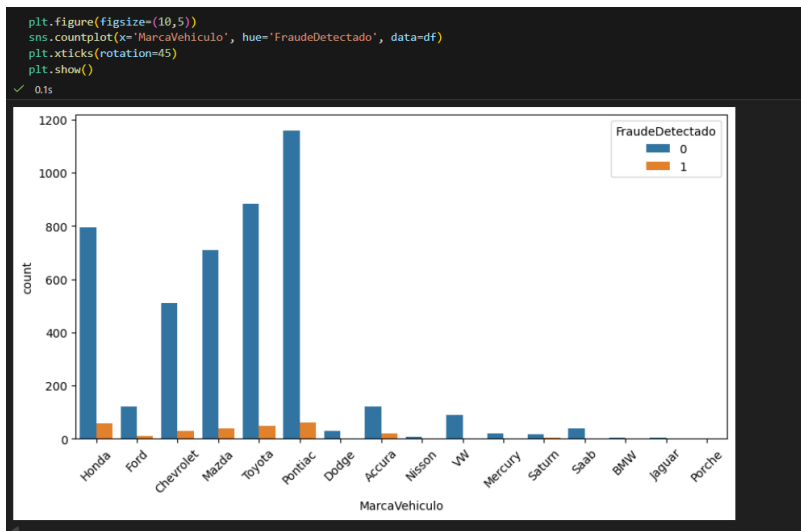
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

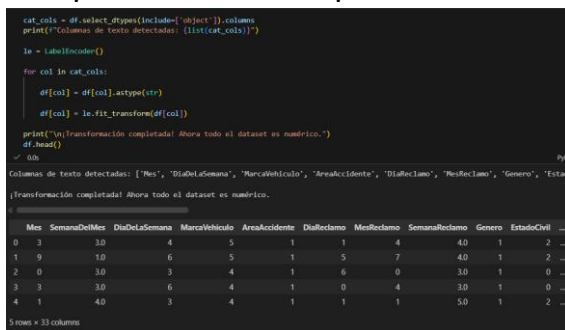
En la cuarta celda se generan estadísticas descriptivas con `df.describe()`, lo cual ofrece una visión general del comportamiento de las variables numéricas, incluyendo promedios, mínimos, máximos y dispersión. Además, se revisan las distribuciones de algunas columnas categóricas relevantes, como “FraudeDetectado” y “MarcaVehiculo”, para comprender qué tan frecuente es cada categoría y cómo se comporta el conjunto de datos desde el punto de vista estadístico.



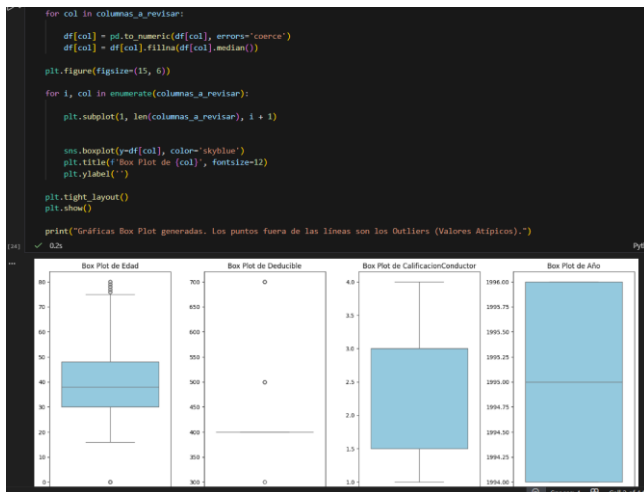
A continuación, en la quinta celda se presenta un gráfico que muestra la distribución de los casos de fraude y no fraude. Esta visualización facilita la identificación de si existe o no un desbalance entre las categorías, lo que es crucial para evaluar si el modelo de machine learning necesitará técnicas adicionales de balanceo.



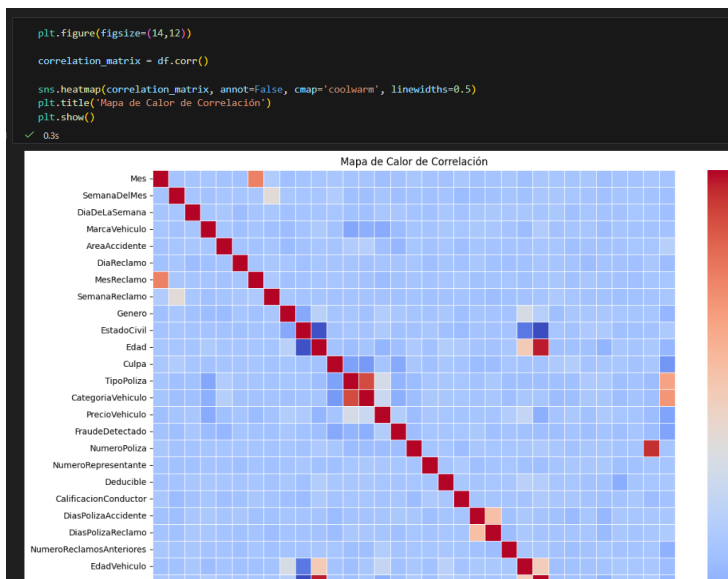
La sexta celda continúa con el análisis visual e incorpora un gráfico que cruza las marcas de vehículo con la variable de fraude. Esto permite observar si existe algún patrón entre el tipo de vehículo y la probabilidad de fraude, lo cual puede resultar útil para interpretar comportamientos sospechosos dentro del dataset.



Posteriormente, la séptima celda se orienta a la detección de valores atípicos mediante gráficos tipo boxplot. Aquí se seleccionan variables numéricas clave para visualizar de manera clara si existen puntos anormales que se alejan de los rangos típicos. La presencia de estos valores puede influir en la calidad del análisis, por lo que esta celda ayuda a prever posibles intervenciones de limpieza o transformación.



En la octava celda se presenta un análisis de correlación entre variables numéricas a través de un mapa de calor. Este gráfico permite observar qué tan relacionadas están las variables entre sí, lo cual es útil para identificar redundancias, posibles problemas de multicolinealidad o relaciones que puedan ayudar en el proceso de modelado.



La novena celda inicia el proceso de modelado de machine learning. En ella se separan las variables predictoras y la variable objetivo “FraudeDetectado”, y posteriormente se divide el dataset en conjuntos de entrenamiento y prueba. Finalmente, se entrena un modelo supervisado con los datos de entrenamiento. Esta celda marca el comienzo de la construcción del modelo predictivo.


```

target = 'FraudeDetectado'
X = df.drop(target, axis=1)
y = df[target]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

model = RandomForestClassifier(n_estimators=100, random_state=42)

print("Entrenando modelo..")
model.fit(X_train, y_train)
print("Modelo entrenado!")

```

✓ 0.2s

Entrenando modelo..
Modelo entrenado!

En la décima celda se hacen las predicciones utilizando los datos de prueba y se genera un reporte de clasificación que muestra métricas como precisión, recall y F1-score. Dichos valores permiten evaluar el desempeño del modelo y determinar si es capaz de identificar correctamente los casos de fraude.

```

# Aquí hacemos las predicciones
y_pred = model.predict(X_test)

# Matriz de Confusión
print("Matriz de Confusión:")
print(confusion_matrix(y_test, y_pred))

print("\nReporte de Clasificación:")
print(classification_report(y_test, y_pred))

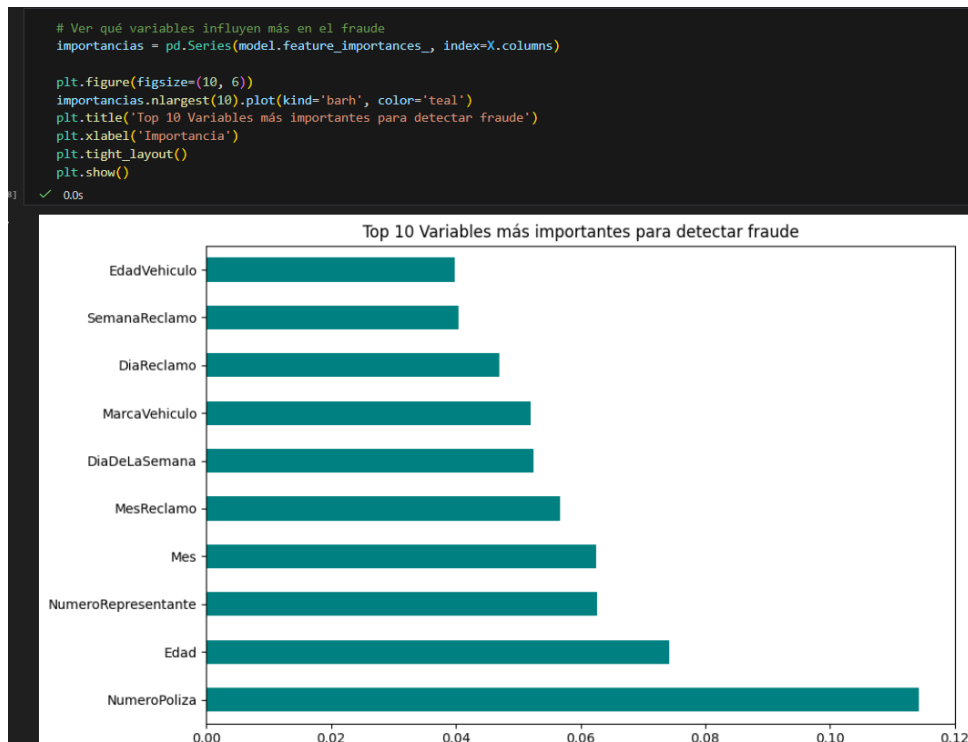
```

✓ 0.0s

Matriz de Confusión:
[[1345 0]
[90 0]]

Reporte de Clasificación:

	precision	recall	f1-score	support
0	0.94	1.00	0.97	1345
1	0.00	0.00	0.00	90
accuracy			0.94	1435
macro avg	0.47	0.50	0.48	1435
weighted avg	0.88	0.94	0.91	1435



La siguiente celda se centra en la interpretación del modelo mediante el cálculo de la importancia de las variables. En esta parte, se genera un gráfico donde se observa cuáles características del dataset contribuyen más a la predicción del fraude. Esto proporciona una primera aproximación a las variables más influyentes.

Para finalizar, como se podrá ver el dataset tuvo que ser cambiado del original sobre mensajes sms spam, debido a que se presentaron problemas al momento de la limpieza en el proyecto anterior, pero se encontró este dataset el cual nos ayudara para los fraudes de seguro de autos, espero mejorar próximamente porque todavía me falta muchísimo para entregar algo de calidad, pero al final se está para aprender y espero entregar cosas mucho mejores en un futuro.